ЗАДАЧА 3: API РЕГИСТРАЦИИ ПОЛЬЗОВАТЕЛЯ

ЗАДАЧА 3.1: REST API ИНТЕРФЕЙС

HTTP Метод: `POST`
URL: `/api/v1/auth/register`

ВХОДНЫЕ ПАРАМЕТРЫ (REQUEST BODY — JSON)

Параметр	Тип	Обязательный	Ограничения	Описание
`firstName`	string	Да	1-50 симв., только буквы и пробелы, регистр любой	Имя пользователя
`lastName`	string	Да	1-50 симв., только буквы и пробелы, регистр любой	Фамилия пользователя
`userName`	string	Да	3-50 симв., латиница, цифры, `_`, `-`, `.`; уникально в системе	Уникальный логин
`password`	string	Да	Мин. 8 симв.; 1 заглавная (A-Z), 1 строчная (a-z), 1 цифра (0-9), 1 спецсимвол (!@#$%^&*)	Пароль пользователя
`recaptchaToken`	string	Да	Токен reCAPTCHA v2/v3, длина 100-2000 симв.	Токен подтверждения reCAPTCHA

Пример JSON:

```
{
  "userId": "550e8400-e29b-41d4-a716-446655440000",
  "userName": "ivan",
  "message": "User registered successfully",
  "createdAt": "2024-01-15T10:30:00Z"
```

УСПЕШНЫЙ ОТВЕТ (201 CREATED)

Таблица выходных параметров:

Параметр	Тип	Обязательный	Ограничения	Описание
`userId`	uuid	Да	Формат UUID v4 (36 симв., шаблон `xxxxxxxx-xxxx-...`)	Уникальный идентификатор пользователя
`userName`	string	Да	3-50 симв., латиница	Имя пользователя из запроса
`message`	string	Да	1-200 симв., фиксированные значения	Сообщение об успехе
`createdAt`	datetime	Да	Формат ISO 8601 (`YYYY-MM-DDTHH:mm:ss Z`)	Дата и время создания аккаунта

Пример ответа:

```
{
  "userId": "550e8400-e29b-41d4-a716-446655440000",
  "userName": "ivan",
  "message": "User registered successfully",
  "createdAt": "2024-01-15T10:30:00Z"
}
```

ОТВЕТЫ С ОШИБКАМИ

400 Bad Request — Ошибка валидации

Параметр	Тип	Обязательный	Ограничения	Описание
`userId`	uuid	Да	Формат UUID v4 (36 симв., шаблон `xxxxxxxx-xxxx-...`)	Уникальный идентификатор пользователя
`userName`	string	Да	3-50 симв., латиница	Имя пользователя из запроса

`message`	string	Да	1-200 симв., фиксированные значения	Сообщение об успехе
`createdAt`	datetime	Да	Формат ISO 8601 (`YYYY-MM-DDTHH:mm:ssZ`)	Дата и время создания аккаунта

Пример JSON:

```
{
  "error": "validation_failed",
  "message": "Password does not meet requirements",
  "fields": {
    "password": "Passwords must have at least one non
alphanumeric character, one digit ('0'-'9'), one uppercase
('A'-'Z'), one lowercase ('a'-'z'), one special character and
Password must be eight characters or longer."
  }
}
```

409 Conflict (Пользователь уже существует)

Параметр	Тип	Обязательный	Ограничения	Описание
`error`	string	Да	Enum: `user_exists`	Тип ошибки
`message`	string	Да	Фиксированное значение	Текст ошибки
`fields`	object	Нет	Объект `{userName: "..."}`	Поле, вызвавшее конфликт

Пример:

```
{
  "error": "user_exists",
  "message": "User exists!",
  "fields": {
    "userName": "Username 'ivan' is already taken"
  }
}
```

400 Bad Request (reCAPTCHA не пройдена)

Пример:

```
{
  "error": "recaptcha_failed",
  "message": "Please verify reCAPTCHA to register!",
  "fields": {}
}
```

500 Internal Server Error (Серверная ошибка)

Параметр	Тип	Обязательный	Ограничения	Описание
`error`	string	Да	Enum: `internal_error`	Тип ошибки
`message`	string	Да	1-200 симв., общий текст	Сообщение для пользователя
`requestId`	string	Нет	Формат `req_[a-z0-9]{9}`	ID запроса для логов

Пример:

```
{
  "error": "internal_error",
  "message": "An unexpected error occurred. Please try again
later.",
  "requestId": "req_abc123xyz"
}
```

СВОДНАЯ ТАБЛИЦА КОДОВ ОТВЕТОВ

Код	Тип	Ситуация	Текст сообщения
201	✅ Success	Регистрация успешна	"User registered successfully"
400	⚠️ Client Error	Ошибка валидации / reCAPTCHA	"validation_failed" / "recaptcha_failed"
409	⚠️ Client Error	Пользователь существует	"User exists!"
500	❌ Server Error	Ошибка сервера	"internal_error"

ПРИМЕР ЗАПРОСА

Блок HTTP:

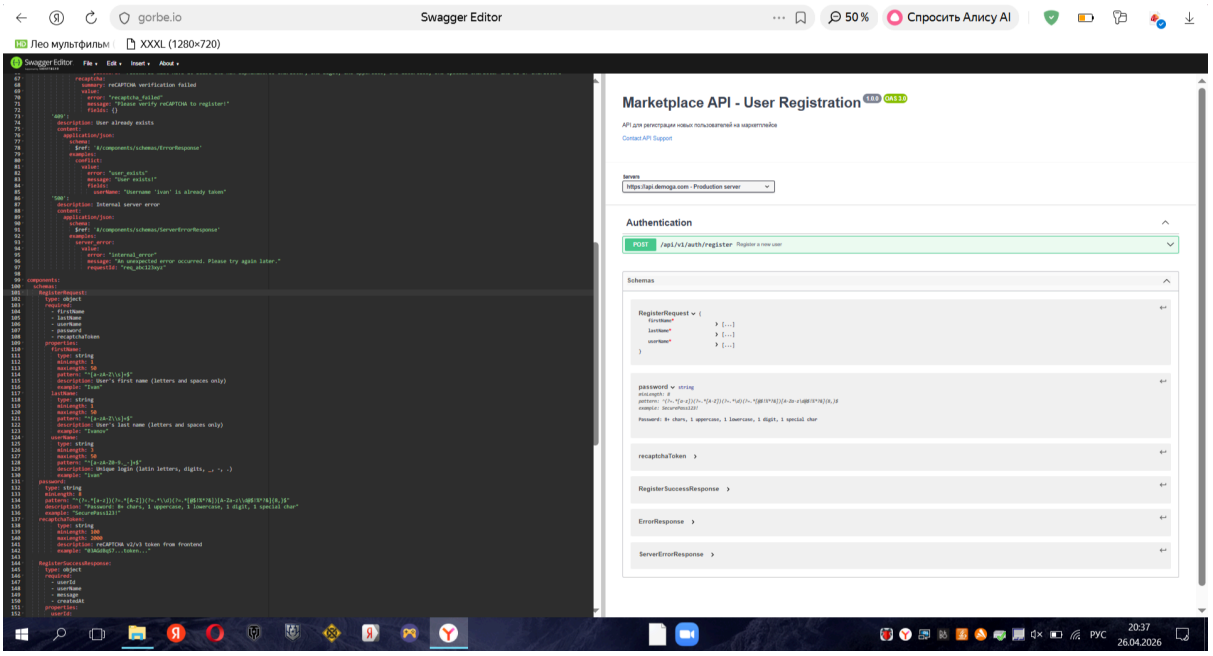
```
POST /api/v1/auth/register HTTP/1.1
Host: demoga.com
Content-Type: application/json

      {
        "firstName": "Ivan",
        "lastName": "Ivanov",
        "userName": "ivan",
        "password": "SecurePass123!",
        "recaptchaToken": "03AGdBqS7...recaptcha_token_here..."
      }
```

ТРЕБОВАНИЯ К ПАРОЛЮ (на основе скриншота)

Пароль должен содержать:

- Минимум 8 символов
- Хотя бы одну букву в верхнем регистре (A-Z)
- Хотя бы одну букву в нижнем регистре (a-z)
- Хотя бы одну цифру (0-9)
- Хотя бы один специальный символ (!@#$%^&*)

Тест в Swagger



OpenAPI Specification (YAML)

```yaml
openapi: 3.0.0
info:
  title: Marketplace API - User Registration
  description: API для регистрации новых пользователей на
маркетплейсе
  version: 1.0.0
  contact:
    name: API Support
    email: support@demoga.com

servers:
  - url: https://api.demoga.com
    description: Production server
  - url: https://staging-api.demoga.com
    description: Staging server

paths:
  /api/v1/auth/register:
    post:
      summary: Register a new user
      description: Создает нового пользователя в системе после
валидации данных и проверки reCAPTCHA
      tags:
        - Authentication
      operationId: registerUser
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/RegisterRequest'
            examples:
              valid:
                summary: Valid request
                value:
                  firstName: "Ivan"
                  lastName: "Ivanov"
                  userName: "ivan"
                  password: "SecurePass123!"
                  recaptchaToken: "03AGdBqS7...token..."
      responses:
        '201':
          description: User registered successfully
          content:
```

```
            application/json:
              schema:
                $ref:
'#/components/schemas/RegisterSuccessResponse'
              examples:
                success:
                  value:
                    userId: "550e8400-e29b-41d4-a716-446655440000"
                    userName: "ivan"
                    message: "User registered successfully"
                    createdAt: "2024-01-15T10:30:00Z"
        '400':
          description: Validation error or reCAPTCHA failed
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ErrorResponse'
              examples:
                validation:
                  summary: Password validation failed
                  value:
                    error: "validation_failed"
                    message: "Password does not meet requirements"
                    fields:
                      password: "Passwords must have at least one
non alphanumeric character, one digit, one uppercase, one
lowercase, one special character and be 8+ characters"
                recaptcha:
                  summary: reCAPTCHA verification failed
                  value:
                    error: "recaptcha_failed"
                    message: "Please verify reCAPTCHA to
register!"
                    fields: {}
        '409':
          description: User already exists
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ErrorResponse'
              examples:
                conflict:
                  value:
                    error: "user_exists"
                    message: "User exists!"
                    fields:
                      userName: "Username 'ivan' is already taken"
        '500':
```

```
          description: Internal server error
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ServerErrorResponse'
              examples:
                server_error:
                  value:
                    error: "internal_error"
                    message: "An unexpected error occurred. Please
try again later."
                    requestId: "req_abc123xyz"

components:
  schemas:
    RegisterRequest:
      type: object
      required:
        - firstName
        - lastName
        - userName
        - password
        - recaptchaToken
      properties:
        firstName:
          type: string
          minLength: 1
          maxLength: 50
          pattern: "^[a-zA-Z\\s]+$"
          description: User's first name (letters and spaces only)
          example: "Ivan"
        lastName:
          type: string
          minLength: 1
          maxLength: 50
          pattern: "^[a-zA-Z\\s]+$"
          description: User's last name (letters and spaces only)
          example: "Ivanov"
        userName:
          type: string
          minLength: 3
          maxLength: 50
          pattern: "^[a-zA-Z0-9._-]+$"
          description: Unique login (latin letters, digits, _, -,
.)
          example: "ivan"
    password:
      type: string
```

```
      minLength: 8
      pattern:
"^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[@$!%*?&])[A-Za-z\\d@$!%*?&]
{8,}$"
      description: "Password: 8+ chars, 1 uppercase, 1 lowercase,
1 digit, 1 special char"
      example: "SecurePass123!"
    recaptchaToken:
          type: string
          minLength: 100
          maxLength: 2000
          description: reCAPTCHA v2/v3 token from frontend
          example: "03AGdBqS7...token..."

    RegisterSuccessResponse:
      type: object
      required:
        - userId
        - userName
        - message
        - createdAt
      properties:
        userId:
          type: string
          format: uuid
          description: Unique user identifier (UUID v4)
          example: "550e8400-e29b-41d4-a716-446655440000"
        userName:
          type: string
          minLength: 3
          maxLength: 50
          description: Registered username
          example: "ivan"
        message:
          type: string
          minLength: 1
          maxLength: 200
          description: Success message
          example: "User registered successfully"
        createdAt:
          type: string
          format: date-time
          description: Account creation timestamp (ISO 8601)
          example: "2024-01-15T10:30:00Z"

    ErrorResponse:
      type: object
      required:
```

```yaml
      - error
      - message
    properties:
      error:
        type: string
        enum:
          - validation_failed
          - user_exists
          - recaptcha_failed
        description: Error type identifier
      message:
        type: string
        minLength: 1
        maxLength: 500
        description: Human-readable error message
      fields:
        type: object
        additionalProperties:
          type: string
        description: Field-specific error details (optional)

  ServerErrorResponse:
    type: object
    required:
      - error
      - message
    properties:
      error:
        type: string
        enum:
          - internal_error
        description: Error type identifier
      message:
        type: string
        minLength: 1
        maxLength: 200
        description: Generic error message for user
      requestId:
        type: string
        pattern: "^req_[a-z0-9]{9}$"
        description: Request ID for support/logs
        example: "req_abc123xyz"
```